the output to public_html and public_html/admin; and add a small .htaccess to each directory that rewrites unknown paths back to index.html so the Angular router, rather than the web server, decides what a deep link means.

The full procedure, including the exact commands and the problems below, is written up in docs/deployment.md so it can be repeated from scratch.

10. Problems Encountered and What They Taught Us

Four things went wrong during deployment. They are recorded here because each one turned out to be a lesson about the layer beneath the one we thought we were working on.

Every emoji became a question mark. The category and organisation icons (a runner, a bowl, a music note, a house) appeared correctly on localhost and as ? on the server. Our first guess was the SQL import, so we added SET NAMES utf8mb4; it did not help. Our second guess was the driver, so we pinned charset: 'utf8mb4' in the mysql2 pool; necessary, but still not enough. The real cause was one level lower: the CREATE TABLE statements never declared a character set, so the tables inherited the database default, and the host's default was utf8mb3 - three bytes per character, which cannot hold a four-byte emoji at all. It had been failing silently since the tables were created, which is why no error was ever raised. The proof was the medical symbol on the wildlife shelter: it sits inside the three-byte range and it was also corrupted, which ruled out 'emoji' as the category of the bug and pointed at the whole character set. The fix was to convert the tables to utf8mb4 and rewrite the affected values; the lasting fix was to pin the character set in the export so the same thing cannot happen again.

The admin site was blank. It worked locally and served a white page on the server, with no 404 and no error message. An Angular build assumes it is mounted at the root unless it is told otherwise, so index.html asked the browser for /main.js while the file had been uploaded to /admin/main.js. The browser received the HTML page again - the .htaccess rewrite - and tried to parse HTML as JavaScript, which fails without a visible message. Rebuilding with --base-href=/admin/ fixed it. The lesson is that a rewrite rule can hide a broken asset path by answering 200 to everything.

Environment variables that were never saved. The API answered 'Access denied for user root@localhost', which is the error you get when no credentials are found and the defaults are used. The variables had been typed into the cPanel form but had not been applied to the application. We switched to a .env file read by dotenv, which is checked into neither the repository nor the zip, and restarted the application: the health endpoint then reported the correct database name. A health endpoint that reports which database it is actually connected to was what made this diagnosable in seconds rather than hours, and we would add one to any API from the start now.

A dependency install that failed once and then worked. Run NPM Install returned a generic error on the first attempt on both accounts and succeeded unchanged on the second. Shared hosts time out under load. We kept a pre-built node_modules archive as a fallback and documented the retry rather than pretending it does not happen.

11. Teamwork

The work was split so that each of us owned a vertical slice of the stack and could explain it end to end in the video, rather than one person writing all the SQL and the other all the HTML.

Table 6 - Division of work.

Part of the brief	Jie Bao (24831941)	Chengyu Gao (24832054)
1. Database	Registrations table, venue coordinates, sample data	Reviewed the schema; booking-date rule
2. REST API	Event CRUD, connection pool, shared validation, delete rule	Registration endpoint, weather proxy, categories and organisations
3. Public site	Angular shell and data layer, Home, Search	Event details, Register page, weather panel
4. Staff site	Event list, delete flow	New and edit screens, shared form component
5. Deployment	Its own cPanel; wrote the deployment guide	Its own cPanel; followed and corrected the guide
Docs and testing	README, deployment notes, video plan, GenAI declaration	Postman collection with assertions, code comments

Git history is the evidence. Each commit message is prefixed with the owner and the part of the brief it belongs to - A1, A2, A3 for Jie's work and B1, B4, B7, B9, B11, B12 for Chengyu's - so the log can be read as a checklist against Table 6. The A3 and B3 commits are deliberately interleaved rather than grouped by author: each person's work was pushed as it was finished, and a commit describes one change rather than

one afternoon. Work spans more than one day, and every commit message explains why the change was made, not just what changed.

To keep the repository honest when one of us could not push from their own machine, we agreed that commits are only ever attributed to the person who wrote the code, and we verified that with git log --pretty='%an %ae' before submitting. The teammate's commits carry his own name and address, not a shared account.

12. My Individual Contribution

I built the database change, the event CRUD endpoints, the shared validation module, the Angular data layer with the home and search pages, the staff event list and its delete flow, and the deployment guide, and I deployed the whole stack to my own cPanel account. The piece of code I am most satisfied with is the delete rule, because it is enforced twice - once in the endpoint, so the staff member gets a sentence they can act on, and once in the foreign key, so the rule cannot be bypassed by anything that talks to the database directly.

The fifteen-minute video follows the same division: I demonstrate the files I own, and I explain the reasoning behind the decisions in them rather than reading the code aloud.

13. Conclusion

Assessment 3 turned a read-only catalogue into an application that accepts data, protects it, and can be operated by somebody who is not a programmer. The parts we are most satisfied with are the ones where a rule lives in exactly one place: the uniqueness of a booking is a UNIQUE key, the deletion rule is a foreign key, and the validation messages come from a single shared module. The parts that taught us most were the failures, all of which happened one layer below where we were looking - a character set in a CREATE TABLE, a base href in a build, an environment variable that was never saved.

If we were to continue, the next steps would be authentication for the staff site, an email confirmation on booking, and pagination on the event list, which is the first thing that will strain once the table grows past a few hundred rows.

14. GenAI Acknowledgement and Chat Log

I acknowledge that I have used GenAI tools to complete this assessment. I used WorkBuddy (an AI coding assistant) to brainstorm the schema change needed for event registration and the set of REST endpoints that would serve both websites; to check grammar and clarity in code comments, the README and the deployment notes; to format documentation; and to review my own code for bugs, within the parameters outlined in the Assessment Brief and by the Unit Assessor.

Table 7 - How GenAI was used, and what it did not do.

Where GenAI was used	What it contributed	What we did ourselves
Brainstorming	Candidate table shapes for registrations; the list of endpoints to consider	Chose UNIQUE (event_id, email); rejected a separate attendee table; chose 409 over 400
Code review	Pointed out that toISOString() returns a UTC date and shifts the calendar day; that the delete check and the delete itself must not be separated by other work	Reproduced both, wrote the fixes, tested them against the live API
Writing support	Grammar and clarity in comments and documentation; table formatting	All design decisions, all final wording, and the structure of the code
Debugging	Suggested character-set causes to rule out during the emoji investigation	Ran the experiments that eliminated the first two explanations and identified the real one

Every design decision recorded in this report is mine and I can explain it, and I am prepared to demonstrate any file I own line by line if asked. The chat log of the sessions is kept with the submission, and a fuller statement is in GENAI-DECLARATION.md in the repository.

Appendix A - Where Each Requirement Is Met

Table 8 - Requirement to evidence.

Requirement	Evidence
New table related to events, with keys	database/charityevents-db.sql - registrations, fk_registration_event, uq_registration_event_email
At least 10 registrations	16 rows across 5 events in the same SQL file
event_db.js updated	api/event_db.js - pool with utf8mb4, transactional helper
Endpoint returning an event with its registrations	GET /api/events/:id - api/routes/events.js
Insert, update and delete endpoints	POST /api/events, PUT /api/events/:id, DELETE /api/events/:id
Delete blocked when bookings exist	409 response plus ON DELETE RESTRICT in the schema
Registration page with validation and confirmation	clientside/src/app/pages/register/

Event page lists bookings, newest first	clientside/src/app/pages/event-detail/, ORDER BY registration_date DESC, registration_id DESC
Register button links to the registration page	Router link to /register?eventId= in event-detail.html
Admin list of all events regardless of status	adminside/src/app/pages/event-list/ with ?status=all
Admin create, update, delete with feedback	event-new, event-edit, event-list
Navigation on every page	adminside/src/app/app.html - shell component
Both websites and the API deployed	Table 5; /api/health returns database: connected
GitHub history with meaningful messages	29 commits, two authors, prefixed with owner and part
Angular used for both websites (extra)	Table 4
Open-Meteo weather API (extra)	api/routes/weather.js and the panel on the event page

Word count (body, excluding tables and code): approximately 3,900 words.